Gestión de Usuarios de un Proyecto

Desde este panel puedes administrar todos los usuarios que tienen acceso a tu aplicación o proyecto creado dentro de la plataforma ROBLE.

Agregar credenciales de acceso

Desde este panel puedes registrar nuevos usuarios en tu proyecto o aplicación. Solo necesitas ingresar su correo electrónico, y el sistema generará automáticamente su UID (identificador único). También puedes asignarles permisos si el proyecto lo permite.

Inhabilitar una cuenta

Puedes desactivar temporalmente el acceso de un usuario sin necesidad de eliminarlo. Esto es útil cuando necesitas suspender el acceso a tu app sin perder los datos asociados al usuario.

Eliminar un usuario

Si deseas remover completamente un usuario del proyecto, puedes hacerlo desde aquí. Esta acción es irreversible, y eliminará todas las credenciales asociadas a ese UID.

Ver listado de usuarios

En esta sección puedes visualizar todos los usuarios registrados en tu proyecto. Para cada usuario verás su correo electrónico, el estado de la cuenta (activo o inhabilitado) y su UID. Esta información es clave para gestionar el acceso y realizar auditorías.

Roles y permisos

ROBLE permite controlar el acceso a los datos mediante un sistema de roles y permisos por tabla. Esto te permite decidir quién puede leer, insertar, actualizar o eliminar registros dentro de tu proyecto.

Rol inicial por defecto

Al crear un proyecto en ROBLE, se genera automáticamente un rol llamado **"user"**. Este rol tiene permisos para **leer** y **actualizar** todas las tablas que existen en ese momento. Sin embargo, estos permisos pueden ser modificados en cualquier momento desde la interfaz.

Flujo recomendado

1. **Crear permisos**: por cada tabla que tenga tu proyecto, define los permisos necesarios: `read`, `insert`, `update`, `delete`.
2. **Crear un rol**: por ejemplo, `editor`, `viewer`, `admin`.
3. **Asignar permisos al rol**: selecciona qué acciones puede realizar ese rol sobre cada tabla.

> **Nota:** Los permisos solo se pueden crear sobre tablas que ya existen en la base de datos. Si luego creas nuevas tablas, deberás definir sus permisos manualmente.

Asignación de roles a usuarios

Desde la sección **Autenticación > Usuarios** puedes editar el rol asignado a cualquier usuario registrado. Basta con seleccionar el nuevo rol desde el selector correspondiente. El nuevo rol tomará efecto en su próximo inicio de sesión.

Ejemplo de flujo

```
1. Crear tabla: "clientes"
2. Crear permisos: clientes:read, clientes:update
3. Crear rol: operador
```

Este sistema permite crear configuraciones de acceso flexibles y seguras para cualquier tipo de proyecto dentro de ROBLE.

TIPOS DE DATOS EN ROBLE

En ROBLE, los tipos de datos son fundamentales para definir la estructura de las tablas y columnas en tu base de datos. Si vas a consumir los endpoints directamente (sin usar las interfaces del frontend), debes usar el valor del **SQL type** para definir el tipo de dato.

- **int2** - Signed two-byte integer
  SQL type: SMALLINT
- **int4** - Signed four-byte integer
  SQL type: INTEGER
- **int8** - Signed eight-byte integer
  SQL type: BIGINT
- **float4** - Single precision float (4 bytes)
  SQL type: REAL
- **float8** - Double precision float (8 bytes)
  SQL type: DOUBLE PRECISION
- **numeric** - Exact numeric of selectable precision
  SQL type: NUMERIC
- **json** - Textual JSON data
  SQL type: JSON
- **jsonb** - Binary JSON data, decomposed
  SQL type: JSONB
- **text** - Variable-length character string
  SQL type: TEXT
- **varchar** - Variable-length character string
  SQL type: VARCHAR
- **uuid** - Universally unique identifier
  SQL type: UUID

SQL type: TIME

timestamp - Date and time (no time zone)
SQL type: TIMESTAMP

timestamptz - Date and time, with time zone
SQL type: TIMESTAMP WITH TIME ZONE

bool - Logical boolean (true/false)
SQL type: BOOLEAN

geography - Geographic point (longitude, latitude)
SQL type: GEOGRAPHY(POINT)

Manipulación de datos

Manipula los datos (registros) dentro de las tablas mediante los endpoints de inserción, consulta, actualización y eliminación. Para poder utilizar estos endpoints, el usuario debe estar autenticado y su rol debe contar con los permisos adecuados para realizar cada operación.

URL Base

```
https://roble-api.openlab.uninorte.edu.co/database/:dbName
```

Ejemplo: https://roble-api.openlab.uninorte.edu.co/database/token_contract_xyz

Endpoints Disponibles

1. POST /:dbName/insert

Inserta múltiples registros (filas) en una tabla específica dentro de la base de datos indicada.

Parámetros:
- dbName (path): Nombre de la base de datos.
- Cuerpo (JSON):
- tableName: Nombre de la tabla destino.
- records: Arreglo de objetos que representan cada fila a insertar, con pares clave-valor donde la clave es el nombre de la columna.

Comportamiento:
- Si el registro no incluye el campo _id, se genera automáticamente un identificador único de 12 caracteres.
- Se validan los tipos de datos para cada columna según la definición de la tabla.

- inserted: registros insertados exitosamente.
- skipped: índices y razones de registros omitidos.

Restricciones:
- El nombre de la tabla debe ser válido.
- Se aplican validaciones estrictas para mantener la integridad de los datos.

Ejemplo de Consumo

Ejemplo de insertMultipleRecords JavaScript

```javascript
import axios from 'axios';

const res = await axios.post(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/insert',
  {
    tableName: 'usuarios',
    records: [
      { name: 'Juan', age: 30 },
      { name: 'Ana', age: 25 }
    ]
  },
  {
    headers: {
      Authorization: 'Bearer TU_ACCESS_TOKEN'
    }
  }
);

console.log(res.data.inserted);
console.log(res.data.skipped);
```

2. GET /:dbName/read?tableName=nombreTabla&campo1=valor1&campo2=valor2...

Parametros:
- dbName (path): Nombre de la base de datos.
- Query:
- tableName: Nombre de la tabla a consultar.
- Otros parámetros opcionales que serán tratados como filtros de igualdad sobre las columnas (ejemplo: estado=activo).

Respuesta:
- Arreglo con los registros que cumplen los filtros indicados.
- Para columnas de tipo geográfico, los datos se retornan en formato legible (WKT).

Restricciones:
- Nombres de tabla y columnas deben ser válidos para evitar inyección.
- Si no se especifica ningún filtro, se retornan todos los registros.

Ejemplo de Consumo

Ejemplo de readRecords JavaScript

```javascript
import axios from 'axios';

const res = await axios.get(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/read',
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN'' },
    params: {
      tableName: 'usuarios',
      age: 30
    }
  }
);

console.log(res.data);
```

3. PUT /:dbName/update

Parametros:
- dbName (path): Nombre de la base de datos.
- Cuerpo (JSON):
- tableName: Nombre de la tabla.
- idColumn: Nombre de la columna que identifica el registro (ejemplo: _id).
- idValue: Valor único para identificar el registro a modificar.
- updates: Objeto con pares columna-valor que representan los campos a actualizar.

Comportamiento:
- Solo se actualizan las columnas indicadas en updates.
- Se respetan los tipos de datos y validaciones definidas en la tabla.
- Para columnas geográficas, acepta valores en formato adecuado.
- Retorna el registro actualizado.

Restricciones:
- El registro identificado por idColumn y idValue debe existir.
- Se valida que los nombres de tabla y columnas sean válidos.

Ejemplo de Consumo

Ejemplo de updateRecord JavaScript

```javascript
import axios from 'axios';

const res = await axios.put(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/update',
  {
    tableName: 'usuarios',
    idColumn: '_id',
    idValue: 'customid1234',
    updates: { age: 31 }
  },
  {
    headers: {
      Authorization: 'Bearer TU_ACCESS_TOKEN'
    }
  }
);
```

4. DELETE /:dbName/delete

Elimina un registro específico de una tabla en la base de datos.

Parámetros:
- dbName (path): Nombre de la base de datos.
- Cuerpo (JSON):
- tableName: Nombre de la tabla.
- idColumn: Nombre de la columna identificadora del registro.
- idValue: Valor que identifica el registro a eliminar.

Comportamiento:
- El registro que coincide con el identificador se elimina permanentemente.
- Retorna el registro eliminado como confirmación.

Restricciones:
- Debe existir un registro con el identificador especificado.
- Validación estricta de nombres para evitar errores e inyección.

Ejemplo de Consumo

Ejemplo de deleteRecord JavaScript

```javascript
import axios from 'axios';

const res = await axios.delete(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/delete',
  {
    headers: {
      Authorization: 'Bearer TU_ACCESS_TOKEN'
    },
    data: {
      tableName: 'usuarios',
      idColumn: '_id',
```

```
console.log(res.data);
```

Gestión de tablas

Gestiona tus tablas y columnas fácilmente con el servicio de base de datos de ROBLE. Se recomienda configurar las tablas directamente desde la interfaz de ROBLE, pero si lo prefieres, también puedes crearlas utilizando estos endpoints a través de la API.

URL Base

```
https://roble-api.openlab.uninorte.edu.co/database/:dbName
```

Ejemplo: https://roble-api.openlab.uninorte.edu.co/database/token_contract_xyz

Endpoints Disponibles

1. POST /:dbName/create-table

Crea una nueva tabla dentro de la base de datos indicada en el parámetro dbName.
Es necesario enviar en el cuerpo de la petición (JSON) el nombre de la tabla (tableName),
una descripción opcional y un arreglo de columnas con las siguientes propiedades para cada columna:
- name: Nombre de la columna (sin espacios ni caracteres especiales).
- type: Tipo de dato SQL válido (ejemplo: varchar(255), integer, boolean).
- defaultValue (opcional): Valor por defecto para la columna.
- isNullable (opcional): Define si la columna acepta valores nulos (default true).
- isPrimary (opcional): Indica si la columna es clave primaria.

Restricciones importantes:
- Se agrega automáticamente una columna _id como identificador único de tipo VARCHAR(12).
- Si no se especifica clave primaria, _id será la clave primaria.
- No se permite que el usuario defina manualmente la columna _id.
- Si alguna columna tiene tipo geográfico (geography o geometry), se habilita automáticamente la extensión PostGIS en la base.

Ejemplo de Consumo

Ejemplo de createTable JavaScript

```javascript
import axios from 'axios';

const res = await axios.post(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/create-tabl
  {
    tableName: 'usuarios',
    description: 'Tabla de usuarios',
    columns: [
      { name: 'id', type: 'integer', isPrimary: true },
      { name: 'email', type: 'varchar' }
    ]
  },
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
  }
);
console.log(res.status);
```

2. PUT /:dbName/update-table/:tableName

Actualiza una tabla existente en la base dbName.
Puedes modificar:
- El nombre de la tabla (renombrarla).
- La descripción o comentario asociado a la tabla.
- La estructura de columnas: agregar nuevas columnas, eliminar columnas existentes (excepto _id), renombrar columnas y cambiar sus tipos de datos.
- La clave primaria: puede modificarse eliminando la actual y asignando una nueva.

El proceso se realiza en una transacción para garantizar atomicidad.
Se validan nombres, tipos y restricciones para evitar inconsistencias.
El campo columns del cuerpo debe incluir las columnas con propiedades:
- name: nuevo nombre.

5. Si hay columnas que no están en el arreglo y existen en la tabla, se eliminan (excepto _id).
Cualquier error revierte toda la operación.

Ejemplo de Consumo

Ejemplo de updateTable JavaScript

```javascript
import axios from 'axios';

const res = await axios.put(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/update-tabl
  {
    tableName: 'usuarios_actualizados',
    description: 'Usuarios con historial de ingreso',
    columns: []
  },
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
  }
);
console.log(res.status);
```

3. DELETE /:dbName/delete-table/:tableName

- Elimina completamente una tabla especificada por tableName en la base de datos dbName.
- Esta operación elimina la tabla y todos sus datos asociados de forma irreversible.
- Usa CASCADE para eliminar también cualquier dependencia.
- Se valida que el nombre de la tabla sea válido.
- En caso de error, se retorna mensaje explicativo.

Ejemplo de Consumo

Ejemplo de deleteTable JavaScript

```javascript
import axios from 'axios';
```

```
  }
);
console.log(res.status);
```

4. GET /:dbName/table-data?schema=public&table=usuarios

Obtiene la definición de columnas y los datos almacenados en una tabla específica.
Parámetros query:
- schema (opcional, default public): esquema donde está la tabla.
- table: nombre de la tabla.

Respuesta incluye:
- columns: listado con nombre y tipo de cada columna.
- rows: datos actuales de la tabla.

Para columnas de tipo geográfico (geography), los valores se retornan en formato WKT legible (ej: POINT(x y)).
Valida nombres y existencia de tabla para evitar errores.

Ejemplo de Consumo

Ejemplo de getTableData — JavaScript

```javascript
import axios from 'axios';

const res = await axios.get(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/table-data?
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
  }
);
console.log(res.data);
```

En el cuerpo de la petición se debe enviar un objeto con:
- tableName: nombre de la tabla.
- column: objeto con definición de la columna, que incluye:
- name: nombre de la columna.
- type: tipo de dato SQL válido.
- defaultValue (opcional): valor por defecto.
- isNullable (opcional): si acepta null.
- isPrimary (opcional): si es clave primaria.

Restricciones importantes:
- No se puede agregar una columna con nombre duplicado.
- No se puede agregar una columna NOT NULL sin valor por defecto si la tabla ya contiene datos.
- Solo se puede definir clave primaria si la tabla no tiene una existente.
- Se validan tipos y nombres para evitar errores.
- El proceso es transaccional para garantizar consistencia.

Ejemplo de Consumo

Ejemplo de addColumn JavaScript

```javascript
import axios from 'axios';

const res = await axios.post(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/add-column'
  {
    tableName: 'usuarios',
    column: {
      name: 'telefono',
      type: 'varchar'
    }
  },
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
  }
);
console.log(res.status);
```

Parámetros:
- tableName (path): tabla objetivo.

En el cuerpo se especifican detalles a modificar:
- oldName: nombre actual de la columna.
- newName: nuevo nombre (si se renombra).
- type: nuevo tipo SQL.
- defaultValue: nuevo valor por defecto o null para eliminar.
- isNullable: si es nullable o no.
- isPrimary: si es clave primaria o no.

Funcionalidad:
- Renombra columna si cambia el nombre.
- Cambia tipo con conversión segura usando USING.
- Cambia nullabilidad y valor por defecto.
- Cambia clave primaria, eliminando la existente si aplica.
- Retorna mensajes de error detallados para problemas comunes.

Ejemplo de Consumo

Ejemplo de updateColumn JavaScript

```javascript
import axios from 'axios';

const res = await axios.post(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/update-colu
  {
    oldName: 'telefono',
    newName: 'telefono_celular',
    type: 'varchar'
  },
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
  }
);
console.log(res.status);
```

En el cuerpo debe especificarse:
- tableName: tabla objetivo.
- columnName: columna a eliminar.

Restricciones:
- No se puede eliminar la columna _id (identificador obligatorio).
- No se puede eliminar una columna que sea clave primaria.
- Se validan nombres para evitar inyección o errores.
- Se retornan errores claros en caso de restricción o fallo.

Ejemplo de Consumo

Ejemplo de dropColumn JavaScript

```javascript
import axios from 'axios';

const res = await axios.post(
  'https://roble-api.openlab.uninorte.edu.co/database/token_project_xyz/drop-columr
  {
    tableName: 'usuarios',
    columnName: 'telefono_celular'
  },
  {
    headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
  }
);
console.log(res.status);
```

API de Autenticación

Aprende a autenticar usuarios utilizando el servicio de autenticación ROBLE.

URL Base

```
https://roble-api.openlab.uninorte.edu.co/auth/:dbName
```

Ejemplo: https://roble-api.openlab.uninorte.edu.co/auth/token_contract_xyz

Endpoints Disponibles

1. POST /:dbName/login

Este endpoint permite autenticar a un usuario en el sistema utilizando su correo electrónico y contraseña. Si las credenciales son válidas, se generan y devuelven un accessToken y un refreshToken para futuras solicitudes autenticadas.

Copy

```
Body:{
  "email": "usuario@correo.com",
  "password": "12345678"
}
```

Copy

```
Response:{
  "accessToken": "jwt-token",
  "refreshToken": "jwt-refresh-token"
}
```

Ejemplo de Consumo

Ejemplo de login JavaScript

```
  email: 'user@example.com',
  password: '123456'
});

console.log(res.data.accessToken);
```

2. POST /:dbName/refresh-token

Permite obtener un nuevo accessToken enviando un refreshToken previamente emitido. Es útil cuando el accessToken ha expirado y se requiere continuar con una sesión activa sin necesidad de volver a iniciar sesión.

Ejemplo de Consumo

Ejemplo de refreshToken JavaScript

```
import axios from 'axios';

const res = await axios.post('https://roble-api.openlab.uninorte.edu.co/auth/toke
  refreshToken: 'jwt-refresh-token'
});

console.log(res.data.accessToken);
```

3. POST /:dbName/signup

Este endpoint registra un nuevo usuario en la base de datos. Se requiere el correo electrónico, una contraseña segura y un nombre completo. Este endpoint debe utilizarse con cuentas reales, ya que se enviará un código de verificación de identidad al correo electrónico registrado.

Ejemplo de Consumo

```
import axios from 'axios';

const res = await axios.post('https://roble-api.openlab.uninorte.edu.co/auth/toke
  email: 'nombre@uninorte.edu.co',
  password: '123456',
  name: 'Nombre Apellido'
});
```

4. POST /:dbName/signup-direct

Permite registrar un usuario directamente sin verificación de correo electrónico. Este endpoint está diseñado para la creación de usuarios de prueba dentro de los proyectos, ya que no se envía ningún código de verificación de identidad.

Ejemplo de Consumo

Ejemplo de signupDirect JavaScript

```
import axios from 'axios';

const res = await axios.post('https://roble-api.openlab.uninorte.edu.co/auth/toke
  email: 'nombre@uninorte.edu.co',
  password: '123456',
  name: 'Nombre Apellido'
});
```

5. POST /:dbName/verify-email

Se utiliza para verificar la dirección de correo electrónico del usuario. El usuario debe proporcionar el código de verificación enviado previamente a su correo electrónico durante el proceso de registro.

Ejemplo de Consumo

```
import axios from 'axios';

const res = await axios.post('https://roble-api.openlab.uninorte.edu.co/auth/toke
  email: 'nombre@correo.com',
  code: '123456'
});
```

6. POST /:dbName/forgot-password

Envía un correo electrónico de recuperación de contraseña al usuario si su dirección de correo está registrada en el sistema. Este correo contiene un token para poder restablecer la contraseña.

Ejemplo de Consumo

Ejemplo de forgotPassword JavaScript

```
import axios from 'axios';

await axios.post('https://roble-api.openlab.uninorte.edu.co/auth/token_contract_x
  email: 'nombre@correo.com'
});
```

7. POST /:dbName/reset-password

Permite al usuario restablecer su contraseña utilizando un token de recuperación válido que recibió por correo. También debe proporcionar una nueva contraseña segura para actualizarla en el sistema.

Ejemplo de Consumo

Ejemplo de resetPassword JavaScript

```
import axios from 'axios';
```

8. POST /:dbName/logout

Este endpoint cierra la sesión del usuario de manera segura. El accessToken debe proporcionarse para invalidar la sesión actual en el sistema y revocar el acceso.

Ejemplo de Consumo

Ejemplo de logout JavaScript

```javascript
import axios from 'axios';

await axios.post('https://roble-api.openlab.uninorte.edu.co/auth/token_contract_x
  headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
});
```

9. GET /:dbName/verify-token

Permite verificar si el accessToken proporcionado es válido y aún no ha expirado. Es útil para validar sesiones activas y controlar el acceso a recursos protegidos.

Ejemplo de Consumo

Ejemplo de verifyToken JavaScript

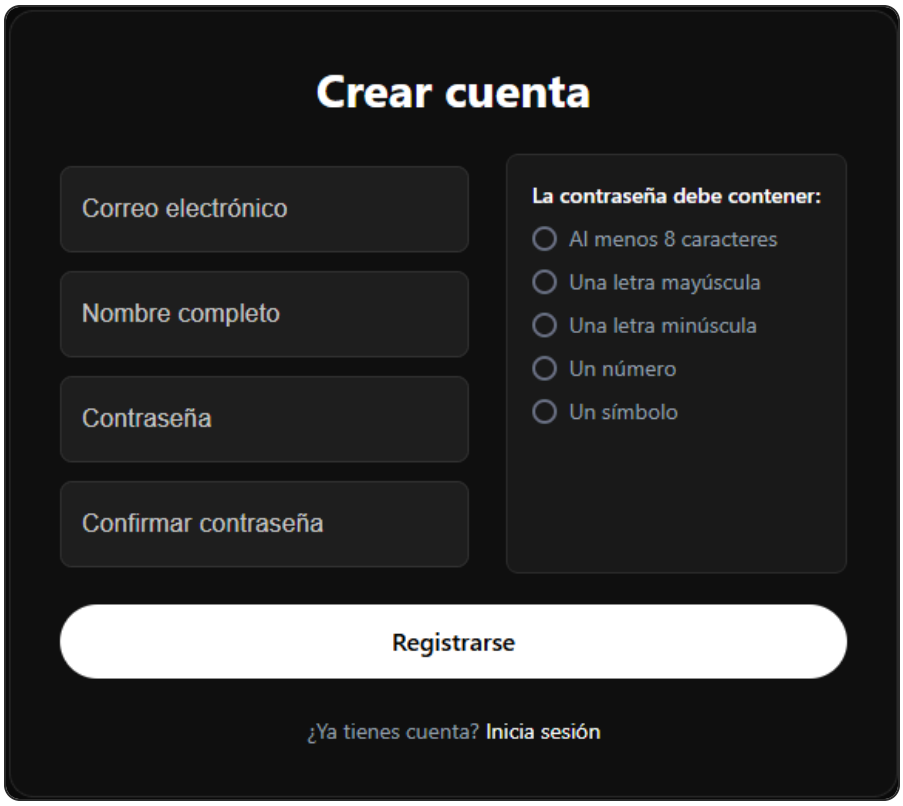
```javascript
import axios from 'axios';

const res = await axios.get('https://roble-api.openlab.uninorte.edu.co/auth/toker
  headers: { Authorization: 'Bearer TU_ACCESS_TOKEN' }
```

Guía para Crear un Proyecto

Sigue estos pasos para crear tu primer proyecto dentro de la plataforma ROBLE y comenzar a consumir la API de autenticación de forma segura.

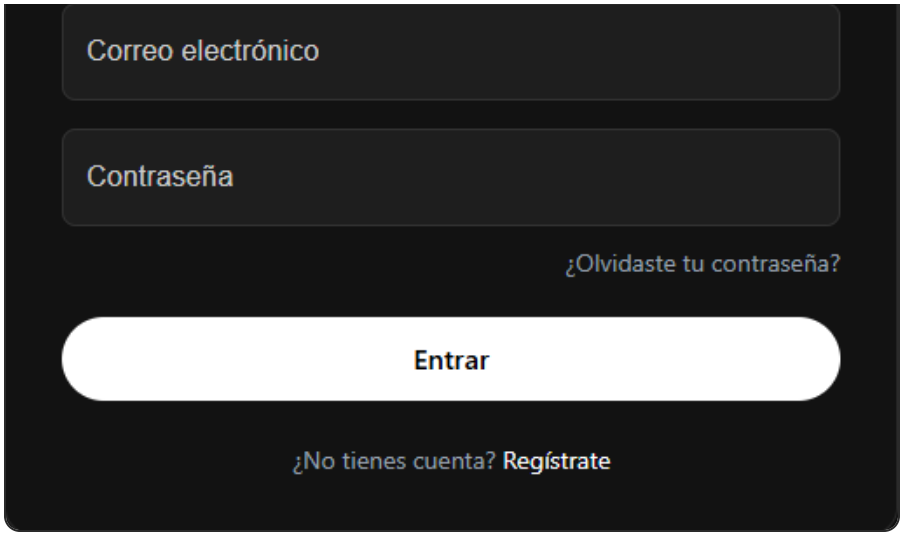
Paso 1: Crea tu cuenta con tu correo Uninorte

Accede a la plataforma ROBLE, ingresa al apartado de crear cuenta y creala tu correo institucional (@uninorte.edu.co). Luego, revisa tu bandeja de entrada, donde recibirás un código de verificación. Ingresa ese código en la plataforma para completar el registro. Este proceso se hace una sola vez para activar tu cuenta de acceso.



Paso 2: Inicia sesión en la plataforma

Una vez que tu cuenta esté verificada, podrás iniciar sesión ingresando nuevamente tu correo institucional y tu contraseña. Serás redirigido al panel principal, donde podrás comenzar a gestionar tus proyectos.

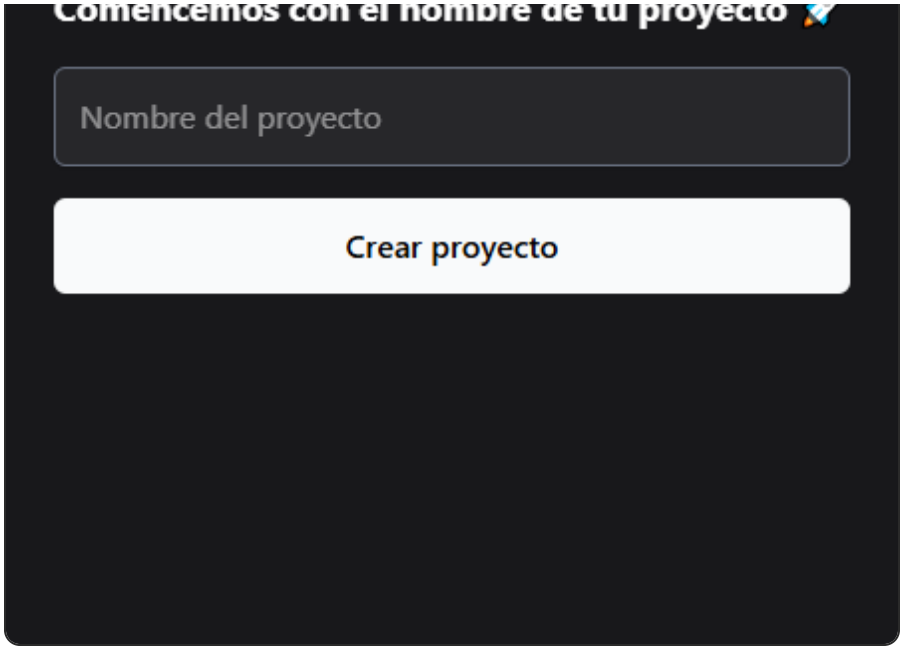


Paso 3: Accede al panel principal

Después del inicio de sesión, verás el panel principal de la aplicación. Aquí aparecerán todos tus proyectos creados. Si es tu primera vez, este panel estará vacío, pero desde aquí puedes crear nuevos proyectos en cualquier momento.

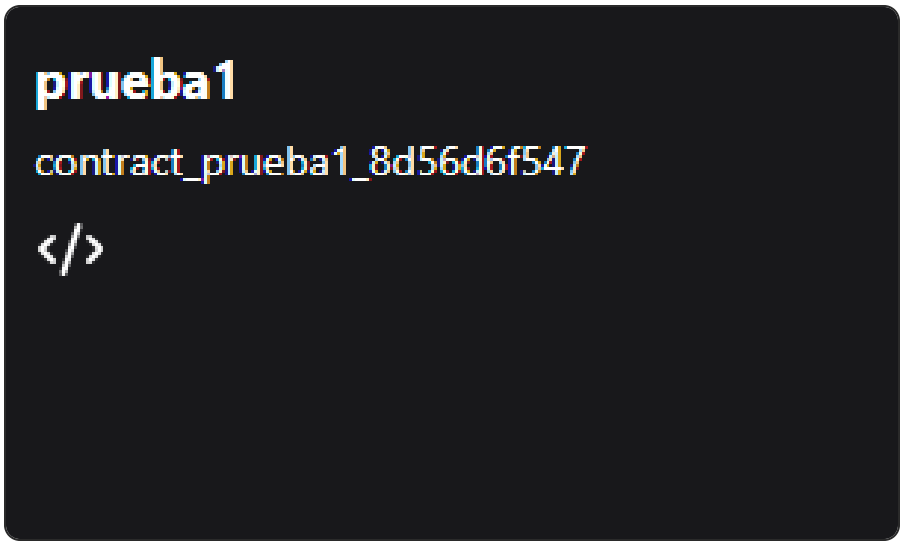
Paso 4: Haz clic en "Crear nuevo proyecto"

Presiona el botón “Crear nuevo proyecto”. Se abrirá una barra lateral donde debes ingresar el nombre de tu proyecto. Asegúrate de que el nombre sea único y representativo. Luego haz clic en el botón “Crear proyecto” para confirmar la creación.



Paso 5: Obtén tu token de autenticación

Después de crear el proyecto, se generará automáticamente un token único para ese proyecto. Este token será necesario para autenticar las solicitudes a la API de autenticación de ROBLE. Guárdalo en un lugar seguro. También podrás verlo nuevamente desde el panel del proyecto.



Paso 6: Gestiona los usuarios de tu proyecto

Tu proyecto aparecerá como una tarjeta dentro del panel principal. Al hacer clic en esta tarjeta, se abrirá el panel de administración del proyecto. Allí podrás ver a los usuarios

INTRODUCCIÓN A ROBLE

ROBLE es una plataforma tecnológica en evolución, diseñada para integrar múltiples servicios digitales bajo una arquitectura abierta, segura y modular. Su objetivo principal es facilitar la gestión, innovación y colaboración tecnológica en entornos académicos y de desarrollo, permitiendo a estudiantes, docentes y desarrolladores acceder a herramientas modernas de forma gratuita, eficiente y personalizada.

En su primera etapa, ROBLE lanza dos servicios principales: un **sistema de autenticación centralizado** y un **servicio de base de datos administrado**. Este último permite a cada proyecto crear y gestionar sus propias tablas, columnas y registros, ya sea desde la interfaz visual o mediante llamadas a la API, sin necesidad de configurar servidores o bases de datos externas.

El servicio de autenticación está basado en estándares modernos como JWT (JSON Web Tokens) y el manejo de sesiones por tokens de acceso y renovación. Puede ser consumido por diversas aplicaciones web y móviles, facilitando su reutilización y adaptabilidad.

El servicio de base de datos permite a cada proyecto crear y administrar sus propias tablas, almacenar registros, y consultar datos directamente desde la API o desde la interfaz gráfica de ROBLE. Este servicio simplifica el desarrollo de aplicaciones que requieren persistencia sin necesidad de administrar servidores o conexiones manuales.

ROBLE no es solo una plataforma funcional, sino también una **iniciativa comunitaria**. Forma parte de **OPENLAB**, un laboratorio de innovación de código abierto que impulsa el desarrollo colectivo de soluciones accesibles, escalables y sostenibles. A través de OPENLAB, se promueve el aprendizaje entre pares, el trabajo colaborativo y la construcción de tecnologías que respondan a necesidades reales dentro y fuera del entorno universitario.

ROBLE es un proyecto comunitario. Si deseas contribuir con código, documentación o ideas, puedes unirte a la comunidad en **OPENLAB**, acceder al repositorio oficial y participar en nuestro canal de Discord. Toda colaboración es bienvenida.